

Image-Domain Gridding on Graphics Processors

Bram Veenboer, Matthias Petschow and John W. Romein

ASTRON (Netherlands Institute for Radio Astronomy)

PO Box 2, 7990 AA Dwingeloo, The Netherlands

Email: {veenboer,petschow,romein}@astron.nl

Abstract—Realizing the next generation of radio telescopes such as the Square Kilometre Array (SKA) requires both more efficient hardware and algorithms than today’s technology provides. The recently introduced image-domain gridding (IDG) algorithm is a novel approach towards solving the most compute-intensive parts of creating sky images: gridding and degriding. It avoids the performance bottlenecks of traditional AW-projection gridding by applying instrumental and environmental corrections in the image domain instead of in the Fourier domain. In this paper, we present the first implementations of this new algorithm for CPUs and Graphics Processing Units (GPUs). A thorough performance analysis, in which we apply a modified roofline analysis, shows that our parallelization approaches and optimizations lead to nearly optimal performance on these architectures. The analysis also indicates that, by leveraging dedicated hardware to evaluate trigonometric functions, GPUs are both much faster and more energy efficient than regular CPUs. This makes IDG on GPUs a candidate for meeting the computational and energy efficiency constraints of future telescopes.

I. INTRODUCTION

At the present time, the world’s largest radio telescope, the *Square Kilometre Array* (SKA) [2], is being developed. It will consist of hundreds of dishes and more than 100,000 antennas; while it enables astronomers “to monitor the sky in unprecedented detail” [2], it can “easily push computing requirements into the exascale domain” [3]. However, even for existing telescopes such as the LOW-Frequency ARray (LOFAR) [4], with in the order of 50,000 antennas grouped into about 50 stations, data processing remains challenging – in particular, when so-called *direction-dependent effects* (DDEs) have to be taken into account [5].

Creating sky images is especially computationally intensive: both efficient algorithms and processing methods are needed to meet the ambitious time and power constraints of instruments such as the SKA [6]. According to a requirements analysis of a pipeline that creates sky images, the subparts of *gridding* and *degriding* are the most dominant parts of the computation [3]. Recently, a novel algorithm for these operations was introduced: *Image-Domain Gridding* (IDG) [7]. Its aim is to address the limitations of traditional methods currently in use – in particular, the challenging DDEs.

This work is supported by the Dutch Ministry of EZ and the province of Drenthe through the ASTRON-IBM Dome grant, the EU FP7 under grant no ICT-610476 (DEEP-ER), and by the NWO through NWO-M (DAS-5 [1]) and Open Competition (617.001.204) grants. The European Commission is not liable for any use that might be made of the information contained in this paper.

The main contributions of this paper are as follows: (1) We present the first implementations of the IDG algorithm on CPUs and GPUs; (2) We present the first efficient degriding implementation on GPUs ever; (3) We apply a thorough analysis of the achieved performance, in which we apply a roofline analysis [8], where not only the hardware limitations, but also the limitations of mathematical functions are taken into account; (4) We provide an assessment of the increasingly important energy efficiency on representative CPU and GPU platforms.

The rest of the paper is organized as follows: In Section II, we provide the necessary background describing the problem that needs to be solved. After discussing related work in Section III, we describe the IDG algorithm in Section IV. In Section V, we show how the algorithm is most efficiently mapped onto state-of-the-art CPUs and GPUs (Intel Xeon, AMD Fury X, and NVIDIA GTX 1080). In Section VI, we present experimental results for all architectures. A thorough analysis uncovers architectural and other features that are most relevant to performance and energy efficiency.

II. BACKGROUND

A radio telescope detects electromagnetic waves that originate from radio sources in the universe. The signals are used, among other things, to construct a map of the sky containing the positions, strengths, and polarization of the sources. As opposed to optical telescopes, which usually consists of a single receiver, modern radio telescopes such as the LOFAR and the SKA are comprised of many small antennas.¹

As shown in Fig. 1, the creation of a sky image requires roughly three steps: (1) the digitized signals from pairs of distinct stations are correlated to produce the so-called *visibilities*, (2) calibration is used to estimate and correct instrument parameters and environmental effects, and (3) an *imaging* step converts partially corrected visibilities into a sky image.

More precisely, for every frequency channel ν , every station of dipole antennas produces two signals (one for each polarization). Multiplying and integrating (correlating) the signals of station pair (p, q) for a short period (in the order of seconds) produces a single visibility, $V_{pq}^{(\nu)} \in \mathbb{C}^{2 \times 2}$. Each visibility is associated with a *uvw*-coordinate, $(u_{pq}^{(\nu)}, v_{pq}^{(\nu)}, w_{pq}^{(\nu)}) \in \mathbb{R}^3$, which describes the distance between the two stations in units of wavelength – thus, it depends on the frequency ν . Ignoring

¹A notable exception is the single dish Five-hundred-meter Aperture Spherical radio Telescope (FAST) [9].

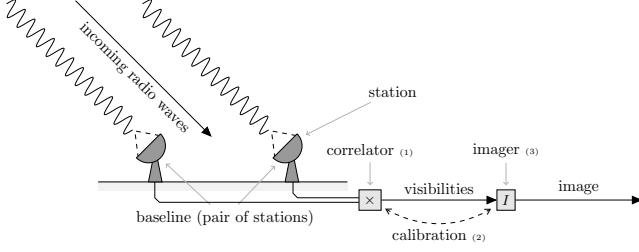


Fig. 1. Incoming radio waves are received by a pair of stations. Since the stations are spaced apart, the signal is out of phase. The visibilities – the correlations of station signals – contain information on the amplitude and phase of the source.

the correction for *direction-independent* effects, the relation between visibilities and sky brightness, $B(l, m) \in \mathbb{R}^{2 \times 2}$, is given by the following measurement equation [10]:

$$V_{pq}^{(\nu)} = \iint_{\ell, m} A_p^{(\nu)} B A_q^{(\nu)H} e^{-2\pi i(u_{pq}^{(\nu)} \ell + v_{pq}^{(\nu)} m + w_{pq}^{(\nu)} n)} d\ell dm, \quad (1)$$

where $\ell, m \in \mathbb{R}$ are the direction cosines of sky coordinates, $n = 1 - \sqrt{1 - \ell^2 - m^2}$, and $A_p^{(\nu)}(\ell, m), A_q^{(\nu)}(\ell, m) \in \mathbb{C}^{2 \times 2}$ describe the aforementioned DDEs. In other words, for a given time and pair of stations, a visibility is a weighted sum of the sky brightness in all directions.

Without the W-terms, $e^{-2\pi i w_{pq}^{(\nu)} n}$, and the A-terms, and after uniformly discretizing the sky coordinates, the relation between visibilities and sky image becomes that of a discrete Fourier transform for non-equidistant data (NDFT). In such a scenario, the sky image can be obtained by a non-uniform fast Fourier transformation (NFFT), which comprises three steps: (1) the non-uniform visibilities are *gridded* onto a uniform grid by applying an operation that corresponds to a convolution, (2) an inverse FFT is applied to the grid, and (3) a correction for the first step is performed [11]. For Eq. (1) however, the gridding step has to take the W-terms and A-terms into account as well – losing the simple NDFT relation. Therefore, we define *gridding* as the means to get, up to a simple correction, the Fourier transform of the discretized sky image from the measured visibilities. Similarly, we define *degridding* as the means to get visibilities from the Fourier transform of the discretized sky image.

In the imaging step (Fig. 2), the measured visibilities are processed independently for different spectral frequency ranges (so-called *image channels*). An image channel corresponds to one or more *data channels*. Imaging typically starts with an empty sky model. After “imaging” (gridding and inverse FFT) the visibilities, one or more bright sources, which mask the more interesting weak sources, are extracted using a variant of the CLEAN algorithm and added to the *sky model* (see [12] and references therein). For these sources, the visibilities are “predicted” (FFT and degridding) and subtracted from the input to reveal fainter sources. This process is repeated until the sky model converges.

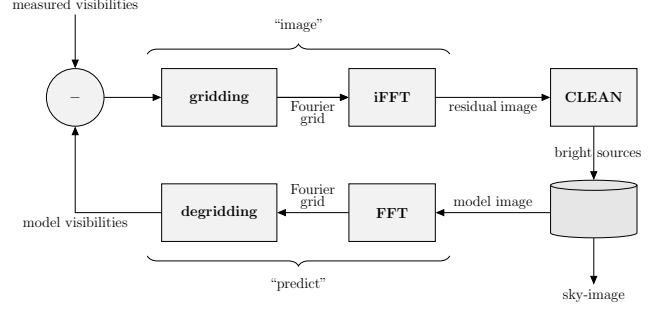


Fig. 2. The imaging step for a single image channel.

III. RELATED WORK

Currently, the most widely used gridding approach is known as *W-projection* [13]. This algorithm corrects for the W-term by means of a convolution in Fourier space, but it does not correct for the DDEs (i.e., the A-terms). However, when antennas are spaced far apart from each other to resolve the high spatial frequencies and observing large fields, the support of the W-terms can become large, making this technique inefficient and memory intensive [14]. The combination of W-projection gridding and *W-stacking* [15], [16] or *W-snapshots* [15] improve on this technique by limiting the support size of W-terms at the cost of using more memory.

The computational challenge increases further when the correction for DDEs is taken into account [14]. The correction can be done in a similar manner to the W-term correction – called *A-projection*. Applying both corrections results in the so-called *AW-projection* gridding [17].

Most state-of-the-art imagers make use of one or more of the various gridding algorithms and their implementations: e.g., CASA [18] and LOFAR’s AWImager [19] use W-projection and AW-projection, while WSClean [16] uses W-stacking only. A-term correction is included in WSClean, but the performance is much lower when this feature is enabled. However, IDG is being integrated into WSClean to improve performance for cases where DDEs are of concern.

The first efficient implementation of W-projection gridding on GPUs has been reported in [20], and has been further improved since [21], [22]. To the best of our knowledge, no gridding and degridding implementation correcting for both W-terms and A-terms has been presented that runs highly efficiently on CPUs and/or GPUs. Even more, no degridding implementation has been presented for GPUs at all. As we show in the following, the IDG algorithm, while efficiently correcting only for the W-term, also allows corrections for DDEs at negligible additional cost. Consequently, it alleviates most of the limitations of traditional AW-projection gridding.

IV. THE IDG ALGORITHM

For every image channel (and, if W-stacking is used, every W-plane), the setting is as depicted in Figs. 3 and 7. Due to the earth’s rotation, measurements associated with one *baseline* (i.e., one pair of stations) draw tracks in the form

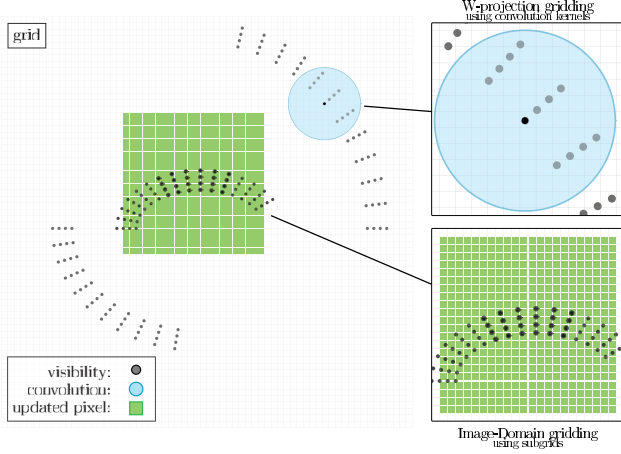


Fig. 3. In traditional W-projection and AW-projection gridding, visibilities are gridded using convolutions in the frequency domain (top-right) as opposed to correcting the W-term and A-term effects in the image domain (bottom right). For the latter, neighboring visibilities are gridded on small ‘subgrids’.

of an ellipsoid in the (u, v) -plane. The sampled (u, v) -plane corresponds to the discrete Fourier transform (DFT) of the sky image (simply called *the grid* hereafter). Every image channel corresponds to C data channels, each baseline is associated with C tracks of T time steps.

W-projection and AW-projection apply a convolution kernel to each of the visibilities (Fig. 3). In W-projection, these kernels depend solely on the w -coordinate associated with the visibility. For AW-projection, these kernels additionally depend on time, channel, and baseline. In practice, the convolution kernels are precomputed on an oversampled grid. The convolution kernels in W-projection or AW-projection gridding form a potentially large multi-dimensional data structure that scales quadratically in size with both the number of pixels in one dimension of the convolution kernel and the oversampling factor. IDG makes use of the classical convolution theorem [23] to perform both W-correction and A-correction in the *image domain*. Consequently, oversampled convolution functions are not needed.

At the center of the algorithm are so-called *subgrids*, which represent low-resolution versions of the sky brightness for a small subset of visibilities (Figs. 3, 5, and 6). These subgrids are placed onto the grid in such a way that they cover a subset of $\tilde{T} \times \tilde{C}$ visibilities and their associated W-term and A-term convolution kernels (see also Section V). Every pixel of the subgrid is then computed as a direct sum of shifted visibilities, as shown in Algorithm 1. Afterwards, both W-term and A-term corrections are applied. Since we have performed the corrections in the image domain, the subgrid has to be Fourier transformed before the result is added to the grid (i.e., four $\tilde{N} \times \tilde{N}$ FFTs per subgrid, one for each of the four combinations of i' and j').

The entire process is illustrated in Fig. 4. Image-Domain Gridding consists of three steps: (1) the visibilities are gridded onto subgrids by the *gridded kernel*, which applies Algorithm 1

Algorithm. 1. Pseudocode that is executed for every subgrid in the gridded kernel. For every evaluation of $\sin(\alpha)$ and $\cos(\alpha)$, 17 real-valued multiply-add operations are performed (one in the evaluation of $f(x, y)$, 16 in the addition to S in Line 10).

```

1  $S = 0;$  //  $\mathbb{C}^{\tilde{N} \times \tilde{N} \times 2 \times 2}$  subgrid
2 for  $y=1, \dots, \tilde{N}$  do
3   for  $x=1, \dots, \tilde{N}$  do
4     for  $t=1, \dots, \tilde{T}$  do // time
5       for  $c=1, \dots, \tilde{C}$  do // channel
6          $\alpha = f(x, y) \cdot g(u(t, c), v(t, c), w(t, c));$ 
7          $\Phi = \cos(\alpha) + i \sin(\alpha);$ 
8         for  $i'=1, 2$  do // polarization
9           for  $j'=1, 2$  do
10             $S(y, x, i', j') += \Phi \cdot \tilde{V}(t, c, i', j');$ 
11          end
12        end
13      end
14    end
15  end
16 end
17 apply_aterm( $S$ );
18 apply_taper( $S$ );
19 store  $S$ ;

```

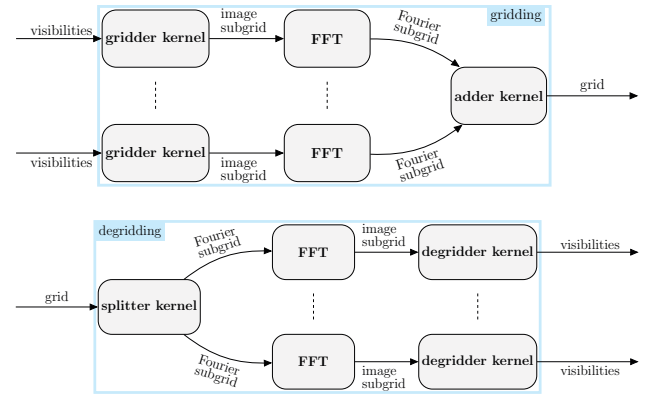


Fig. 4. The Image-Domain routines are drop-in replacements for the gridding and degridding steps in Fig. 2.

for every subgrid; (2) the subgrids are Fourier-transformed. This step will be referred to as the *subgrid FFTs*; (3) the transformed subgrids are added to the grid by the *adder kernel*.

The *degridding* step is similar to the gridding step, but proceeds in reverse order: First, subgrids are extracted from the grid by the *splitter kernel*, then every subgrid is Fourier transformed by an inverse FFT, and finally the associated visibilities are predicted by the *degridded kernel*, which is similar to the gridded kernel and shown in Algorithm 2.

The minimum size of the subgrids follows from the size of the W-term and A-term convolutions kernels and the tapering function that are used to reduce aliasing (such as a prolate spheroidal wave function, which is used in our case). For the LOFAR telescope, subgrids as small as 24×24 pixels are found to provide sufficient accuracy [7]. Furthermore, larger subgrids (say 64×64) can be used to limit the number of planes in W-stacking. At the cost of more computations, this reduces

Algorithm. 2. Pseudocode that is executed for every subgrid or work element in the degridder kernel.

```

1  $\tilde{V} = 0;$  //  $\mathbb{C}^{\tilde{T} \times \tilde{C} \times 2 \times 2}$  visibilities
2 apply_taper( $S$ );
3 apply_aterm( $S$ );
4 for  $t=1, \dots, \tilde{T}$  do // time
5   for  $c=1, \dots, \tilde{C}$  do // channel
6     for  $y=1, \dots, \tilde{N}$  do // subgrid
7       for  $x=1, \dots, \tilde{N}$  do
8          $\alpha = -f(x, y) \cdot g(u(t, c), v(t, c), w(t, c));$ 
9          $\Phi = \cos(\alpha) + i \sin(\alpha);$ 
10        for  $i'=1, 2$  do // polarization
11          for  $j'=1, 2$  do
12             $\tilde{V}(t, c, i', j') += \Phi \cdot S(y, x, i', j')$ 
13          end
14        end
15      end
16    end
17  end
18 end
19 store  $\tilde{V}$ ;

```

the memory footprint, enabling the creation of larger, higher-resolution images.

V. ARCHITECTURES & OPTIMIZATION

In this section, we demonstrate how the IDG algorithm is efficiently mapped onto modern CPUs and GPUs. The CPU class is represented by Intel's Xeon processors family, while the GPU class is represented by an AMD Fury X (Fiji architecture) and Nvidia GTX 1080 (Pascal architecture).

A. The execution plan

Before any execution, we specify the subgrid locations and associate visibilities to the subgrids in the form of an *execution plan*. If $\mathcal{V} = \{V_{pq}(t, c) : \forall t \forall c\}$ denotes the visibilities from all baselines, the positions of the subgrids induce a partitioning $\mathcal{V} = \mathcal{V}_1 \cup \mathcal{V}_2 \cup \dots \cup \mathcal{V}_n$. This process of positioning the subgrids to cover all visibilities is implemented in the form of a greedy algorithm. As depicted in Fig. 5, not only the visibilities need to be covered by the subgrids, but also the support of their associated AW-projection convolution kernels [7]. Thus, for each baseline, starting with time step 1 and having $\tilde{C}$ channels that can be covered by an $\tilde{N} \times \tilde{N}$ subgrid, we include as many time steps as possible (each with $\tilde{C}$ channels) until they cannot be covered anymore. In that case, we create new subgrids (with new positions on the grid) to cover the remaining channels and repeat the process at time step $\tilde{T} + 1$.

We might additionally require that $\tilde{T} \leq \tilde{T}_{\max}$ (where $\tilde{T}_{\max}$ is architecture specific and depends on the other observation parameters) to limit the maximal number of time steps that are associated with a single subgrid. Such an approach keeps the amount of computation to be performed for each subgrid comparable, and the required memory limited.

We call each subgrid S_j (including its metadata such as its position in the grid) together with its associated visibilities $\mathcal{V}_j$ (including *uvw*-coordinates) a *work element*. The set of

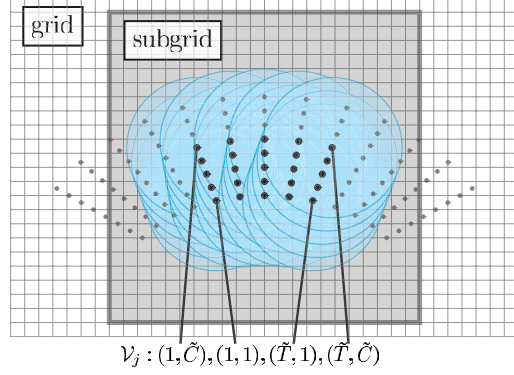


Fig. 5. A subset of visibilities ($\mathcal{V}_j$, black dots), including their associated AW-projection convolution kernels (blue circles), is covered by a subgrid.

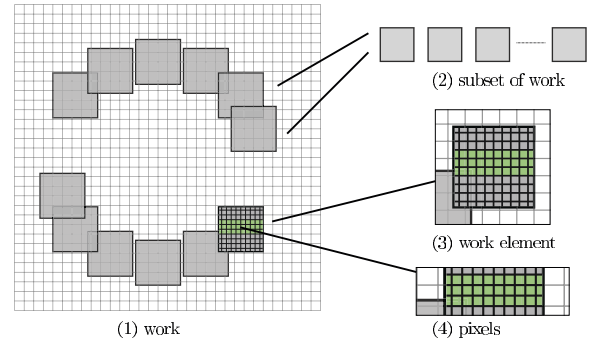


Fig. 6. Work division within IDG: (1) The *work* is partitioned into (2) subsets. (3) Every subset contains work elements, represented by individual subgrids. (4) For gridding, the smallest unit of computation is a single pixel. For degridding, the smallest unit of computation is a single visibility.

all n work elements is called the *work*, and is generated by the execution plan. Subsets of the work are processed by the gridder and degridder kernels, using Algorithm 1 and Algorithm 2 for every work element, respectively. This work division hierarchy is illustrated in Fig. 6, and we will refer to it later to show how the algorithm is mapped differently onto the two distinct architectures classes, CPUs and GPUs.

B. CPU

The CPU class is represented by Intel's Xeon server processor family. These CPUs are available in a wide variety of models and vary in core-counts (up to 24 cores) and clock speed (up to 3.4 Ghz). The last generations (Haswell-EP and Broadwell-EP) have a similar underlying architecture: they implement the AVX2 instruction set and have, for each core, 64 KB of L1 Cache and 256 KB L2 Cache. Furthermore, all cores share a L3 cache of 2.5 MB per core. The clock speed is dynamically adjusted depending on the workload.

Every core contains two floating point execution units (FPUs), which both support vector fused multiply-add (FMA) instructions, with 8-element single-precision SIMD vector width. To fully utilize all CPU cores and execution units, IDG needs to exploit both thread-level and vector-level parallelism.

Finding the right granularity for both levels is key to good performance and scalability. We now detail our strategy for various subparts: *a) gridder kernel*, *b) degridder kernel*, *c) subgrid FFTs*, and *d) adder and splitter*.

a) Gridder kernel: The gridder kernel (as all other kernels implemented in C++) is executed for every subset of the work. It distributes the work elements over all logical cores using OpenMP. In other words, each thread computes a subset of the subgrids according to Algorithm 1.

The most important performance optimizations are the following: (1) We load and transpose visibility data of $T_B \leq \tilde{T}$ time steps and $C_B \leq \tilde{C}$ channels at a time into memory-aligned arrays to allow for non-strided data access (T_B and C_B are a platform-specific optimization parameters; that is, the computation is performed in batches). At this moment, for a better mapping to the instruction set, we also separate the real and imaginary part of the operands. (2) The sine/cosine-computations (Line 7 of Algorithm 1) are precomputed for the entire batch of visibilities with either Intel’s Short Vector Math Library (SVML) or Vector Math Library (VML), whichever is faster. (3) We vectorize the channel loop (Line 5) by writing the computation in the form of a SIMD reduction illustrated in Algorithm 3.

Algorithm. 3. The *reduction* clause instructs the compiler to vectorize over channels. Each vector loop executes 16 FMAs.

```

1 #pragma omp simd reduction(+:...)
2 for c=1,...,C do // channel
3   Re(pix11) += Re(vis11[c]) * Re(Φ[c]);
4   Im(pix11) += Re(vis11[c]) * Im(Φ[c]);
5   Re(pix11) -= Im(vis11[c]) * Im(Φ[c]);
6   Im(pix11) += Im(vis11[c]) * Re(Φ[c]);
7
8   // [... same for pix12 and pix21]
9
10  Re(pix22) += Re(vis22[c]) * Re(Φ[c]);
11  Im(pix22) += Re(vis22[c]) * Im(Φ[c]);
12  Re(pix22) -= Im(vis22[c]) * Im(Φ[c]);
13  Im(pix22) += Im(vis22[c]) * Re(Φ[c]);
14 end

```

The vectorization works best when the number of channels is a multiple of the SIMD vector width, as otherwise the *remainder*(C_B , SIMD_WIDTH) channels will be processed using masked vector instructions. This implies that wider vectors will not necessarily result in higher performance. Finally, we aid compiler-assisted vectorization in the remainder of the kernel by using *runtime compilation*, i.e., we only compile the kernel when the parameters are known at runtime.

b) Degridder kernel: We distribute the work in the same manner as in the gridder kernel: each thread processes a subset of the work by applying Algorithm 2 to every work element. In other words, each thread computes the visibilities for a subset of the subgrids. The kernel optimizations are similar to the optimizations for the gridder kernel. A notable difference is that we apply vectorization over pixels (Line 7 of Algorithm 1). This works best when the number of pixels in one row of the subgrid is a multiple of the SIMD vector

width.

c) Subgrid FFTs: All subgrids are Fourier-transformed before adding them to the grid and after splitting the subgrids from the grid. This is an embarrassingly parallel process and most efficiently done by using a highly-optimized math library such as Intel’s Math Kernel Library (MKL).

d) Adder and splitter: As subgrids might partially overlap in the grid, for the adder, parallelization over subgrids would imply prohibitive synchronization costs. Instead, we parallelize over the rows of the grid to avoid concurrent updates of the same pixels. For the splitter, overlapping subgrids are not a problem as the data from the grid is read-only. Therefore, we parallelize over subgrids.

C. GPU

We implemented IDG on GPUs in their native programming language: CUDA for NVIDIA GPUs [24] and OpenCL for AMD GPUs [25]. Both of these standards include a runtime system and a set of C/C++ extensions to operate the GPU. Apart from syntactic differences, these standards offer the same basic functionality. In the remainder of this paper we adhere to the CUDA terminology.

Compared to CPUs, high-end GPUs such as Nvidia’s GTX 1080 have a much higher core count (2560 cores) and run at a lower clock speed (1733 Mhz). The cores are organized in smaller groups of 64 cores, which are called *Streaming Multiprocessors* (SMs). Every SM contains a register file, load-store units, dedicated caches and a software-managed cache. GPUs are connected to a host machine using PCIe and have dedicated device memory. The device memory is limited in size (8 GB), but the memory bandwidth is higher than on a dual-socket Xeon-based system (e.g. 320 GB/s versus 136 GB/s). Since device memory resides in a separate memory space, data needs to be copied explicitly over the PCIe bus to and from the device memory.

We use triple-buffering to prevent the GPU from being idle during data transfers. To this end we start three threads on the host, allocate buffers on the device (threefold) and create three different CUDA *streams*: one for host-to-device memory transfers, one for kernel execution, and one for device-to-host memory transfers.

On any GPU, threads are organized in a three-level hierarchy: grid-level, block-level, and warp-level. When a kernel is executed, the GPU spawns a *grid* of *thread-blocks* (both of user-specified dimensions) and dispatches the thread-blocks onto the SMs.

This programming model requires a different parallelization strategy compared to the CPU implementation. We detail our parallelization and optimization strategies in the following paragraphs.

a) Gridder kernel: The gridder kernel is launched once for every subset of the work. The number of thread blocks is set according to the cardinality of the subset. The number of threads in a thread block is an architecture-specific optimization parameter.

TABLE I
THE THREE ARCHITECTURES USED IN THIS COMPARISON.

Model	Type	Architecture	Transistors (nm)	Clock (GHz)	Core config ^{a)} (#FPUs)	Peak (TFlop/s)	Mem size (GB)	Mem bw (GB/s)	TDP (W)
Intel Xeon E5-2697v3	CPU	Haswell-EP	22	2.60 ^{b)}	2×14×2×8 = 448	2.78 ^{b)}	≤1536	136	290
AMD R9 Fury X	GPU	Fiji	28	1.050	1×64×1×64 = 4096	8.60	4	512	275
NVIDIA GTX 1080	GPU	Pascal	16	1.733 ^{b)}	1×40×2×32 = 2560	9.22 ^{b)}	8	320	180

^{a)} #ICs × #compute units × FPU instructions/cycle × vector size

^{b)} turbo mode enabled

In order to process a single work element, a thread block executes Algorithm 1. We apply the following optimizations: (1) Line 2 and 3 have been collapsed and threads are mapped onto pixels based on their position in the thread block. (2) Since the size of input data scales linearly with $\tilde{N}$ and $\tilde{C}$, we process the input data in batches. Thus, we transpose and prefetch the batch of data into fixed-size shared memory buffers. (3) While iterating through the batch data, every thread accumulates all contributions to its current pixel in registers. Only when the threads have finished the entire batch, they write the pixel value to device memory using coalesced memory accesses. (4) The sine/cosine computation in Line 7 is performed using regular `sin` and `cos` function calls. During compilation, we pass `-use_fast_math` to the CUDA compiler to enable the use of optimized sine/cosine functions. These functions have a maximum error of 2 units in the last place (ulps) [26], which is sufficient for IDG [7].

b) Degridder kernel: Similar to the gridded kernel, the degridder kernel is launched once for every subset of the work. For the processing of a single work element, a more sophisticated parallelization strategy is used in which threads have two different roles. We apply the following optimizations to Algorithm 2: We collapse Lines 4 and 5 and map threads onto (a subset of) visibilities based on their position in the thread block – the first mapping. We collapse Lines 6 and 7 and also map the threads onto pixels – the second mapping. According to the second mapping, every thread loads a pixel value from device memory, applies the tapering function and the A-term correction (Lines 2 and 3), and stores the result into shared memory. Furthermore, every thread evaluates $f(x, y)$ in Line 8 and stores the result in shared memory. At this point, threads return to their first role (according to the first mapping) and perform the remainder of the algorithm: they evaluate $g()$ in Line 8 and execute Lines 9 to 12. Next, the procedure is repeated for the next batch of pixels. During the entire process, threads keep their first role and update their associated visibility in registers. Just like in the gridded kernel, the sine/cosine-evaluations (Line 9) are executed using optimized math functions.

When the number of threads is lower than the $\tilde{T} \times \tilde{C}$ visibilities associated with the subgrid, the procedure described above is repeated: threads are mapped onto a next batch of visibilities and an iteration over all pixels is performed until the entire work item is processed.

c) Subgrid FFTs: We use the cuFFT and cFFT libraries to compute Fourier transformations for the CUDA and OpenCL implementations, respectively.

d) Adder and splitter: We have two options to add all subgrids onto the grid: (1) perform the operation on the GPU, or (2) copy the subgrids into host memory and perform the operation on the CPU. The second option is only required when dealing with large images that no longer fit into the GPU device memory. For the first option, a relatively simple kernel, where every thread-block atomically adds all pixels to the grid was found most efficient. In CUDA we use the built-in `atomicAdd` function twice, to add both the real and imaginary components of the pixel to the grid. Atomic operations on floating point are not available using the current AMD OpenCL runtime, so we resorted to a loop that updates grid values using `atomic_cmpxchg`.

Unlike the adder kernel, no synchronization is required for the splitter kernel. For every subgrid we copy the relevant part of the grid into the subgrid buffer.

VI. RESULTS

In this section, after describing the experimental setup, we analyze IDG's performance for each architecture in detail. Additionally, we demonstrate that IDG is far more suited for GPUs: in relative terms, i.e., same peak performance, GPUs are almost four times faster and about six times more energy efficient than CPUs; in absolute terms, GPUs are even an order of magnitude faster and more energy efficient than CPUs.

A. Experimental setup

All experiments were executed on the DAS-5 cluster [1] using the hardware listed in Table I. We refer to the three systems as HASWELL (a dual-socket system with Haswell-EP processors), FIJI (a system hosting an AMD R9 Fury X GPU), and PASCAL (a system hosting a NVIDIA GTX 1080 GPU).

For HASWELL, we used Intel's compiler (version 17.0.0) together with MKL (version 2017.0.0); for FIJI, we used the AMD APP SDK 3.0 OpenCL runtime and GPU driver version 1800.11; for PASCAL, we used CUDA 8.0.27 and GPU driver 367.35. All computations are performed in single-precision floating-point operations (flops). We therefore omit the term "single-precision" from here on.

As performance is data dependent (the *uvw*-coordinates determine the subgrid configuration and, hence, the computational intensity within the gridded and degridder kernels, the

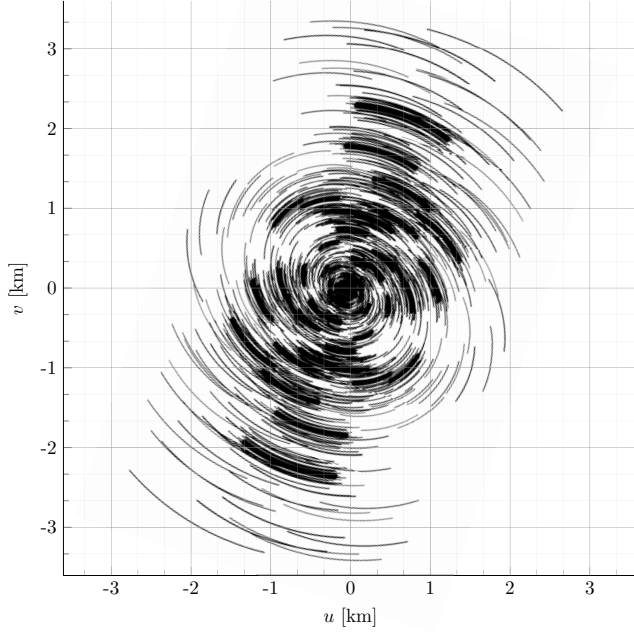


Fig. 7. (u, v) -plane for our test data set.

synchronization in the adder, and the total amount of subgrids to be Fourier-transformed), we created a representative benchmark using proposed antenna coordinates for the SKA1-Low telescope – the phase 1 subset of the SKA covering the low frequency spectrum [27]. The data set has the following parameters: 150 stations (11,175 baselines), T equals 8,192 time steps (at 1 second integration time) and C equals 16 channels; the A-terms (for simplicity, all set to identity) are updated every 256 time steps; the subgrids and grid are 24×24 pixels and 2048×2048 pixels in size, respectively. The (u, v) -plane for this data set is illustrated in Fig. 7.

B. Performance comparison

In Fig. 8 and Fig. 9 we present, respectively, the execution time and throughput (measured in MVisibilities/s) for the single imaging cycle shown in Fig. 2. Both GPUs complete the task almost an order of magnitude faster than HASWELL. For all architectures, runtime is dominated by the gridded and degridded kernels (in all cases, more than 93% of the runtime). Since the impact on execution time of all other kernels is limited, we focus on the gridded and degridded kernels for the remaining analysis.

In Fig. 10 we show performance for the two kernels in the form of a roofline plot [8], where an operation (an *op*) is defined as one of the following: $+$, $-$, $*$, $\sin()$, $\cos()$. (The dashed lines are described later.) As about 17 out of 18 operations in Algorithms 1 and 2 are FMAs, the plot can be interpreted as TFlop/s, if the sine/cosine evaluations are ignored. However, we include the sine/cosine evaluations in the definition of an

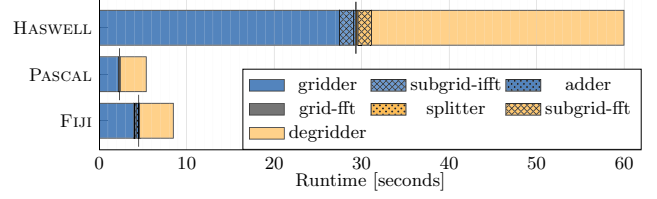


Fig. 8. Distribution of runtime for one full imaging cycle.

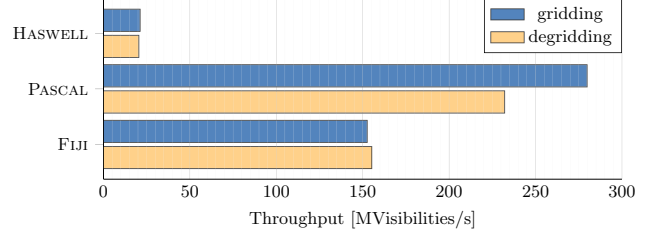


Fig. 9. Throughput for gridding and degridding.

operation for two reasons: (1) These mathematical functions are implemented differently in the different architectures and we have to treat them as black boxes. (2) on PASCAL these operations are supported by *dedicated* hardware [26]. Despite this definition, the peak performance measured in TOP/s is still only achieved when using FMAs exclusively.

According to their operational intensity, both kernels fall in the compute-bound regime of the roofline graph. This means that given an optimal implementation, the number of operations performed per byte moved from/to memory is sufficiently high to attain the peak performance of the architecture. In practice, however, only on PASCAL does the measured performance approach the theoretical peak performance: 74% and 55% for the gridded and degridded kernel, respectively. In contrast, on HASWELL (and FIJI), both kernels perform significantly lower than the peak: 19% (47%) and 18% (49%) for the gridded and degridded kernels, respectively.

C. Performance analysis

On HASWELL and FIJI, since the measured performance is far from the peak performance, the roofline analysis indicates that our implementations are bound by something else.

1) *The HASWELL and FIJI case:* Defining sine/cosine evaluations as operations leads to a number of complications: (1) their performance highly depends on the mathematical library that is used; (2) their performance depends on the settings such as the accuracy and the number of values computed at once; (3) their performance can also be dependent on the input values. On HASWELL, without changing the performance significantly, we use SVML for which performance is independent of the number of values computed. Furthermore, we select medium accuracy (maximum of 4 ulps error), and ensure input values in the range of $[-10^4, 10^4]$. On FIJI, the native AMD math library is used.

As the gridded and degridded kernel perform 17 real-valued FMA instructions for every sine/cosine evaluation (see Al-

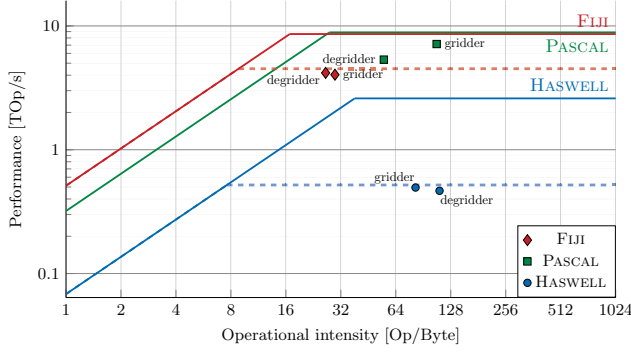


Fig. 10. Roofline analysis: one operation is $+$, $-$, $*$, $\sin()$ or $\cos()$. Peak performance is only achieved if non-masked FMA instructions (two operations) are used exclusively. While the operation count is known exactly, the data movement is measured.

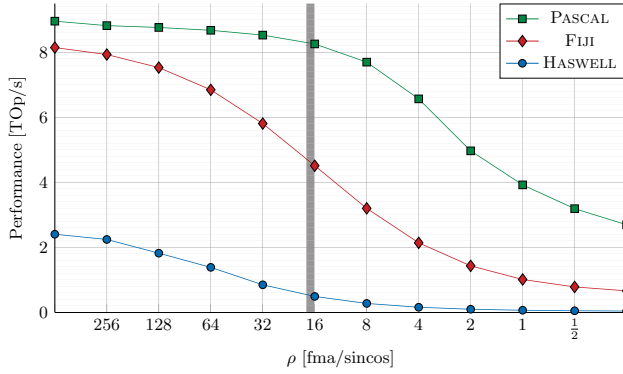


Fig. 11. Operation throughput for various mixes of FMA instructions and sine/cosine evaluations. The shaded line at $\rho = 17$ indicates the operation mix as found in the gridder and degridder kernels.

gorithms 1 and 2), this situation is similar to an instruction imbalance in a traditional roofline analysis, where peak performance is only achieved if an equal number of multiplications and additions are performed in the form of FMAs. In order to determine an upper performance bound for our workload, we benchmarked the performance for various ratios of

$$\rho = \frac{\text{number of FMAs}}{\text{number of sincos}}.$$

In this definition we use the fact that, for our kernels, sine and cosine are always evaluated on the same argument, $\text{sincos}(\alpha)$, which is more efficient than computing sine and cosine separately. The result of the benchmark is shown in Fig. 11.

PASCAL's SMs contain a number of special-function units (SFUs) that implement the computation of both transcendental functions and interpolation in hardware [28]. Since sine/cosine is handled in a separate processing queue, the performance of PASCAL stays high when ρ decreases. In contrast, on FIJI, the sine/cosine evaluations are performed by the same ALUs that also compute the FMAs, at a quarter of the rate [29]. Consequently, a more significant performance degradation is observed for small values of ρ . Performance for the software-

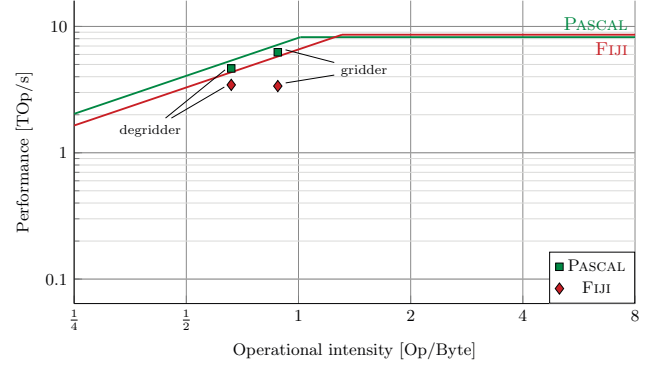


Fig. 12. Roofline graph with (measured) shared memory bandwidth as memory roof. The operational intensity is computed with respect to the amount of bytes transferred from shared memory.

based sine/cosine library used on HASWELL shows a similar degradation for small values of ρ .

Given these insights, we establish new upper bounds for the peak Ops/s for the gridder and degridder kernels in Fig. 10 (dashed lines). These bounds correspond to the values in Fig. 11 for $\rho = 17$. Now, the gridder and degridder kernels are close to optimal, given the limitations of hardware *and* the supporting mathematical library; unfortunately, we cannot use the full computational capacity of HASWELL and FIJI without algorithmic changes.

2) *The PASCAL case:* Although sine/cosine evaluations are supported in hardware, the measured kernel performance is below the theoretical peak performance (Fig. 10). To further investigate the cause, we create a second roofline graph in Fig. 12 – this time with respect to shared memory instead of device memory.

Both kernels are close to the shared memory bandwidth limit. Interestingly, the kernels on FIJI are also relatively close to hitting the shared memory bandwidth limit.

Thus, while the performance of IDG on HASWELL and FIJI is bound by sine/cosine evaluations, on PASCAL, it is bound by the bandwidth of shared memory.

D. Energy efficiency comparison

We use LIKWID [30] to measure the energy usage of both the package (CPU cores and caches) and the DRAM for HASWELL. For FIJI and PASCAL, we measure the energy consumption of the full PCIe device, using PowerSensor [31]. PowerSensor comprises a combination of custom device and host library, which provides power measurements at high time resolution and enables us to analyze energy usage for individual compute kernels. Since GPUs require a host system to operate, we additionally measure energy consumption for the CPU package and the DRAM.

The total energy consumed during the execution of a single imaging cycle is shown in Fig. 13. As most of the time is spent in the gridder and degridder kernels (Fig. 8), the bulk of the energy is naturally also spent in these kernels. Thus, also in terms of energy efficiency, the GPUs outperform the CPU

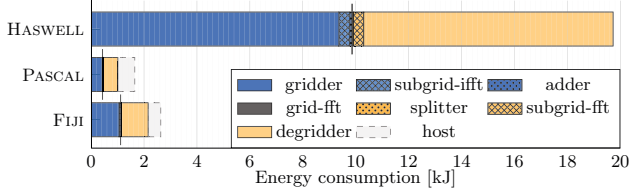


Fig. 13. Distribution of energy consumption for a single imaging cycle.

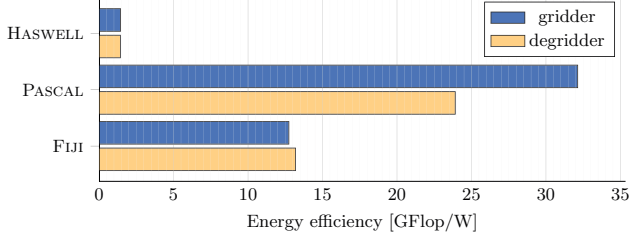


Fig. 14. Energy efficiency for the gridder and degridder kernels.

by an order of magnitude. This is even true when the power consumption of the host is taken into account.

In Fig. 14 we present the energy efficiency of the individual kernels. PASCAL is the most energy-efficient GPU: for the gridder and degridder kernels, it achieves 32 and 23 GFlop/W, respectively. Second, but still with about 13 GFlop/W for both kernels, comes FIJI. HASWELL lags far behind the GPUs, achieving only about 1.5 GFlop/W.

E. Comparison with W-projection

For the same data set, we compare IDG with the W-projection algorithm introduced in [20]. The latter is one of the fastest implementations of W-projection gridding and we refer to it as WPG from now on. For our tests, WPG uses an oversampling factor of 8 for the W-kernels, and their computation is not included in the measurement of the execution time.

The maximum size of the W-kernels, $N_W \times N_W$ pixels, is determined by the observation settings (the instrument, the field of view, the target location, etc.) [17], [19]. For the LOFAR telescope, although the maximum W-kernel can be as large as 500×500 pixels for pure W-projection gridding, N_W is usually much smaller (on average, N_W is 30) [19]. In practice, WPG and IDG are used in conjunction with W-stacking to limit N_W to small values in all situations (e.g., $N_W \leq 16$). Allowing large W-kernels, however, reduces the need to have additional W-planes (copies of the grid, where a subset of the visibilities are gridded onto), which can be prohibitively memory consuming for high-resolution images.

With the above in mind, in Fig. 15 we present the performance on PASCAL for various values of N_W , ranging from the relatively large to the relatively small. While for large W-kernel sizes performance is comparable, for smaller kernels, IDG even outperforms WPG. However, very recently improvements have been introduced to WPG that can increase its performance from roughly 28% of the peak floating-point

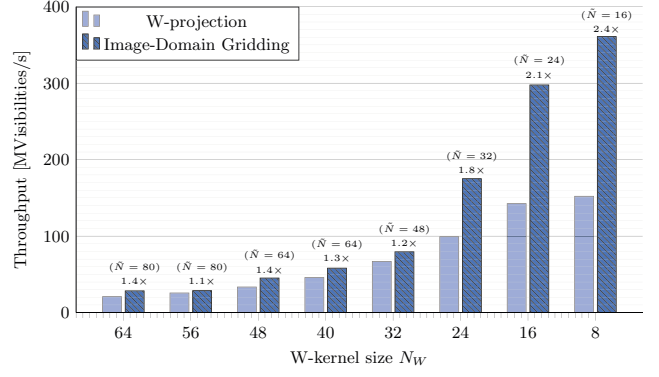


Fig. 15. Throughput of WPG [20] and IDG for various W-kernel sizes. In practice, $N_W \leq 24$ is more common than larger values of N_W . The numbers directly above the bars represent the speedup of IDG compared to WPG. $\tilde{N}$ denotes the subgrid size used within IDG.

performance (which we measured in our tests) to 55% in the best case [22]. Even in this scenario, IDG's performance is comparable to W-projection gridding.

More importantly, IDG achieves such performance while being able to correct for DDEs; in this case, the additional cost to IDG is negligible, while including this feature to the W-projection gridding (so called AW-projection) “requires significantly more instructions and bandwidth for loading the [convolution kernels], because they are dependent on time, frequency, polarization and possibly baseline, and are not separable” [22]. As a consequence, AW-projection gridding becomes much slower and memory consuming than W-projection gridding. Although, to draw definite conclusions, a complete comparison of traditional gridding and IDG is needed, the results suggest that IDG on GPUs is a very significant achievement to address the problems of AW-projection gridding: IDG achieves the performance of the simpler W-projection gridding, while providing the functionality of the more challenging AW-projection gridding.

VII. CONCLUSIONS

We presented the first implementations of the novel Imaging-Domain Gridding algorithm on CPUs and GPUs; we demonstrated that our parallelization and optimization strategies result in close to optimal performance on three distinct architectures: Intel Xeon (Haswell), AMD Fiji, and Nvidia Pascal. In particular for the latter two, the graphics processors, the IDG algorithm maps elegantly onto the underlying hardware. Due to hardware support for sine/cosine evaluations, on Nvidia GPUs, our code achieves up to 74% of the floating-point peak performance and over 30 GFlop/W, while only being limited by shared memory bandwidth. Consequently, IDG on GPUs is a candidate to meet the demanding computational and energy efficiency constraints imposed by future telescopes such as the Square Kilometre Array (SKA).

REFERENCES

- [1] H. Bal *et al.*, “A medium-scale distributed system for computer science research: Infrastructure for the long term,” *IEEE Computer*, vol. 49, pp. 54–63, May 2016.
- [2] The SKA Organisation, “Square Kilometre Array,” 2017.
- [3] R. Jongerius *et al.*, “An end-to-end computing model for the Square Kilometre Array,” *IEEE Computer*, vol. 47, pp. 48–54, Sept. 2014.
- [4] M. P. van Haarlem *et al.*, “LOFAR: The LOw-Frequency ARray,” *Astron. Astrophys.*, vol. 556, 2013.
- [5] R. J. van Weeren *et al.*, “LOFAR facet calibration,” *The Astrophysical Journal Supplement Series*, vol. 223, p. 2, Mar. 2016.
- [6] E. Vermij *et al.*, “Challenges in exascale radio astronomy: Can the SKA ride the technology wave?,” *International Journal of High Performance Computing Applications*, vol. 29, pp. 37–50, Feb. 2015.
- [7] B. van der Tol and B. Veenboer, “Image Domain Gridding,” 2017. In preparation.
- [8] S. Williams, A. Waterman, and D. Patterson, “Roofline: An insightful visual performance model for multicore architectures,” *Communications of the ACM*, vol. 52, pp. 65–76, Apr. 2009.
- [9] R. Nan *et al.*, “The five-hundred-meter aperture spherical radio telescope (FAST) project,” *Int. J. Mod. Phys. D*, vol. 20, pp. 989–1024, June 2011.
- [10] O. M. Smirnov, “Revisiting the radio interferometer measurement equation,” *Astronomy & Astrophysics*, vol. 531, p. A159, July 2011.
- [11] L. Greengard and J.-Y. Lee, “Accelerating the nonuniform fast Fourier transform,” *SIAM Review*, vol. 46, pp. 443–454, Jan. 2004.
- [12] U. Rau *et al.*, “Advances in calibration and imaging techniques in radio interferometry,” *IEEE Proceedings*, vol. 97, pp. 1472–1481, Aug. 2009.
- [13] T. J. Cornwell, K. Golap, and S. Bhatnagar, “The noncoplanar baselines effect in radio interferometry: The W-projection algorithm,” *IEEE J. Sel. Topics Signal Process.*, vol. 2, pp. 647–657, Oct. 2008.
- [14] A. Scaife, “SDP memo: The SDP imaging pipeline,” tech. rep., SKA Science Data Processor Consortium, 2016.
- [15] T. J. Cornwell, M. A. Voronkov, and B. Humphreys, “Wide field imaging for the Square Kilometre Array,” *Proc. SPIE*, vol. 8500, Aug. 2012.
- [16] A. R. Offringa *et al.*, “WSClean: an implementation of a fast, generic wide-field imager for radio astronomy,” *Mon. Not. R. Astron. Soc.*, vol. 444, pp. 606–619, Aug. 2014.
- [17] S. Bhatnagar *et al.*, “Correcting direction-dependent gains in the deconvolution of radio interferometric images,” *Astron. Astrophys.*, vol. 487, pp. 419–429, Aug. 2008.
- [18] S. Jaeger, “The Common Astronomy Software Application (CASA),” in *Astronomical Data Analysis Software and Systems XVII* (R. W. Argyle, P. S. Bunclark, and J. R. Lewis, eds.), vol. 394 of *ASP Conference Series*, pp. 623–627, Aug. 2008.
- [19] C. Tasse *et al.*, “Applying full polarization A-projection to very wide field of view instruments: An imager for LOFAR,” *Astron. Astrophys.*, vol. 553, p. A105, May 2013.
- [20] J. W. Romein, “An efficient work-distribution strategy for gridding radio-telescope data on GPUs,” in *Proceedings of the 26th ACM international conference on Supercomputing*, pp. 321–330, June 2012.
- [21] D. Muscat, *High-Performance Image Synthesis for Radio Interferometry*. PhD thesis, University of Malta, 2014.
- [22] B. Merry, “Faster GPU-based convolutional gridding via thread coarsening,” *Astron. Comput.*, vol. 16, pp. 140–145, July 2016.
- [23] R. Bracewell, *The Fourier Transform and Its Applications*. McGraw Hill, 1965.
- [24] D. Luebke, “CUDA: Scalable parallel programming for high-performance scientific computing,” in *Proc IEEE Int Symp Biomed Imaging*, pp. 836–838, May 2008.
- [25] J. E. Stone, D. Gohara, and G. Shi, “OpenCL: A parallel programming standard for heterogeneous computing systems,” *IEEE Comput. Sci. Eng.*, vol. 12, no. 1-3, pp. 66–73, 2010.
- [26] *NVIDIA CUDA Programming Guide*. 2007.
- [27] B. Mort, “A simple interferometer baseline coordinate generator.” GitHub: SKA-ScienceDataProcessor/uvwsim, 2015.
- [28] S. Oberman and M. Siu, “A high-performance area-efficient multifunction interpolator,” in *17th IEEE Symposium on Computer Arithmetic*, pp. 272–279, 2005.
- [29] Advanced Micro Devices, Inc., *Southern Islands Series Instruction Set Architecture*. 2014.
- [30] J. Treibig, G. Hager, and G. Wellein, “LIKWID: A lightweight performance-oriented tool suite for x86 multicore environments,” *Proceedings of the International Conference on Parallel Processing*, pp. 207–216, 2010.
- [31] J. W. Romein and B. Veenboer, “PowerSensor: a tool to analyze energy efficiency,” 2017. Under review.